좋다. 이건 **한 번에 감 잡히게** 가야 한다.

예시 테이블 두 개를 먼저 보자.

1. 원본 테이블

A 테이블: 학생

```
student_id | name
-----------+------
1          | 철수
2          | 영희
3          | 민수
```

B 테이블: 점수

```
student_id | score
-----------+------
1          | 90
2          | 80
4          | 70
```

- `1`, `2` 는 양쪽 다 있음
- `3` 은 학생 테이블에만 있음
- `4` 는 점수 테이블에만 있음

---

2. JOIN 종류별 결과

---

(1) JOIN = INNER JOIN

보통 `JOIN` 이라고만 쓰면 대부분 **INNER JOIN**이다.

뜻

양쪽에 공통으로 있는 것만 가져옴

```
SELECT *
FROM students s
JOIN scores sc
  ON s.student_id = sc.student_id;
```

결과:

```
student_id | name | student_id | score
-----------+------+------------+------
1          | 철수 | 1          | 90
2          | 영희 | 2          | 80
```

쉽게 말하면

- 서로 **겹치는 애들만** 데려옴

기억법

- **INNER = 교집합**

---

(2) LEFT JOIN

뜻

왼쪽 테이블은 무조건 다 살리고,
오른쪽에서 맞는 것만 붙임

```
SELECT *
FROM students s
LEFT JOIN scores sc
  ON s.student_id = sc.student_id;
```

결과:

```
student_id | name | student_id | score
-----------+------+------------+------
1          | 철수 | 1          | 90
2          | 영희 | 2          | 80
3          | 민수 | NULL       | NULL
```

쉽게 말하면

- 왼쪽인 `students` 는 전부 나옴
- 오른쪽 `scores` 에 없으면 `NULL`

왜?

- 철수(1) → 점수 있음
- 영희(2) → 점수 있음
- 민수(3) → 점수 없음 → `NULL`

기억법

- **LEFT = 왼쪽은 무조건 살린다**

(3) RIGHT JOIN

뜻

오른쪽 테이블은 무조건 다 살리고,
왼쪽에서 맞는 것만 붙임

```
SELECT *
FROM students s
RIGHT JOIN scores sc
  ON s.student_id = sc.student_id;
```

결과:

```
student_id | name | student_id | score
-----------+------+------------+------
1          | 철수 | 1          | 90
2          | 영희 | 2          | 80
NULL       | NULL | 4          | 70
```

쉽게 말하면

- 오른쪽인 `scores` 는 전부 나옴
- 왼쪽 `students` 에 없으면 `NULL`

왜?

- 1 → 학생 정보 있음
- 2 → 학생 정보 있음
- 4 → 학생 정보 없음 → 이름이 `NULL`

기억법

- **RIGHT = 오른쪽은 무조건 살린다**

---

(4) FULL JOIN

뜻

양쪽 테이블 전부 다 살림

```
SELECT *
FROM students s
FULL JOIN scores sc
  ON s.student_id = sc.student_id;
```

결과:

```
student_id | name | student_id | score
-----------+------+------------+------
1          | 철수 | 1          | 90
2          | 영희 | 2          | 80
3          | 민수 | NULL       | NULL
NULL       | NULL | 4          | 70
```

쉽게 말하면

- 왼쪽만 있어도 나옴
- 오른쪽만 있어도 나옴
- 없는 쪽은 `NULL`

기억법

- **FULL = 양쪽 전부**

---

(5) CROSS JOIN

뜻

조건 없이 전부 다 조합함

```
SELECT *
FROM students s
CROSS JOIN scores sc;
```

결과:

```
name | score
-----+------
철수 | 90
철수 | 80
철수 | 70
영희 | 90
영희 | 80
영희 | 70
민수 | 90
민수 | 80
민수 | 70
```

학생 3명 × 점수 3개 = **9행**

쉽게 말하면

- 학생 한 명마다 점수 모든 행을 다 붙임
- 그냥 **경우의 수 전체 생성**

기억법

- **CROSS = 곱하기**

---

3. 진짜 쉬운 비유

학생 명단과 시험 점수표를 대조한다고 생각해보자.

INNER JOIN

"**양쪽에 다 있는 사람만 보자**"

LEFT JOIN

"**학생 명단은 전부 보자**. 점수 있으면 붙이고, 없으면 비워둬"

RIGHT JOIN

"**점수표는 전부 보자**. 학생 명단에 없으면 이름 비워둬"

FULL JOIN

"**둘 다 다 보자**. 한쪽만 있어도 버리지 말자"

CROSS JOIN

"학생 한 명마다 점수표 전체를 다 붙여서 **모든 조합** 만들어보자"

---

4. 그림 느낌으로 정리

A = students

`1, 2, 3`

B = scores

`1, 2, 4`

INNER

`1, 2`

LEFT

`1, 2, 3`

RIGHT

`1, 2, 4`

FULL

`1, 2, 3, 4`

CROSS

`A의 모든 행 × B의 모든 행`

---

5. 시험장에서 제일 쉽게 외우는 법

- **JOIN(=INNER)** : 겹치는 것만
- **LEFT** : 왼쪽 다
- **RIGHT** : 오른쪽 다
- **FULL** : 둘 다 다
- **CROSS** : 전부 조합

---

6. PostgreSQL 예시 문법 한 번에

```sql
-- INNER JOIN
SELECT *
FROM students s
INNER JOIN scores sc
  ON s.student_id = sc.student_id;

-- LEFT JOIN
SELECT *
FROM students s
LEFT JOIN scores sc
  ON s.student_id = sc.student_id;

-- RIGHT JOIN
SELECT *
FROM students s
RIGHT JOIN scores sc
  ON s.student_id = sc.student_id;

-- FULL JOIN
SELECT *
FROM students s
FULL JOIN scores sc
  ON s.student_id = sc.student_id;

-- CROSS JOIN
SELECT *
```

---

7. 제일 중요한 포인트 하나

`INNER` / `LEFT` / `RIGHT` / `FULL` 은 보통 **ON 조건**이 있다.

```
ON s.student_id = sc.student_id
```

근데 `CROSS JOIN` 은 조건 없이 그냥 붙인다.

그래서 데이터가 많으면 행 수가 폭발한다.

예:

- 100행 × 1000행 = 100,000행

---

8. 네가 진짜 이렇게 기억하면 됨

JOIN
→ 겹치는 것만

LEFT
→ 왼쪽 기준

RIGHT
→ 오른쪽 기준

FULL
→ 양쪽 다 챙김

CROSS
→ 다 곱함

원하면 다음엔 내가 이걸 **벤다이어그램 느낌 + null 포함 그림표**로 더 직관적으로 정리해줄게.

natural join은 그냥 join이랑 같은거냐?

아니다. **같지 않다.**

차이

1. 그냥 JOIN

보통은 이렇게 쓴다.

```
SELECT *
FROM A
JOIN B
ON A.id = B.id;
```

이건 **내가 직접 조건을 써주는 join**이다.

즉:

- 어떤 컬럼끼리 비교할지 내가 정함
- `ON` 절로 명확하게 지정

---

2. NATURAL JOIN

이건 **이름이 같은 컬럼을 DB가 알아서 찾아서 자동으로 연결**한다.

```
SELECT *
FROM A
NATURAL JOIN B;
```

즉:

- `ON` 을 안 씀
- 두 테이블에 **같은 이름의 컬럼**이 있으면
- 그 컬럼들을 기준으로 자동 조인

예시

A

```
id | name
---+-----
1  | 철수
2  | 영희
```

B

```
id | score
---+------
1  | 90
2  | 80
3  | 70
```

그냥 JOIN

```
SELECT *
FROM A
JOIN B
ON A.id = B.id;
```

NATURAL JOIN

```
SELECT *
FROM A
NATURAL JOIN B;
```

이 경우는 결과가 비슷할 수 있다.
왜냐면 공통 컬럼 이름이 `id` 하나뿐이라서.